

Optical Flow: Predicting movement with the RAFT model

Note

Try on [Colab](https://colab.research.google.com/github/pytorch/vision/blob/gh-pages/main/_generated_ipynb_notebooks/plot_optical_flow.ipynb) or `go to the end` to download the full example code.

Optical flow is the task of predicting movement between two images, usually two consecutive frames of a video. Optical flow models take two images as input, and predict a flow: the flow indicates the displacement of every single pixel in the first image, and maps it to its corresponding pixel in the second image. Flows are (2, H, W)-dimensional tensors, where the first axis corresponds to the predicted horizontal and vertical displacements.

The following example illustrates how torchvision can be used to predict flows using our implementation of the RAFT model. We will also see how to convert the predicted flows to RGB images for visualization.

```
import numpy as np
import torch
import matplotlib.pyplot as plt
import torchvision.transforms.functional as F

plt.rcParams["savefig.bbox"] = "tight"

def plot(imgs, **imshow_kwargs):
    if not isinstance(imgs[0], list):
        # Make a 2d grid even if there's just 1 row
        imgs = [imgs]

    num_rows = len(imgs)
    num_cols = len(imgs[0])
    _, axs = plt.subplots(nrows=num_rows, ncols=num_cols, squeeze=False)
    for row_idx, row in enumerate(imgs):
        for col_idx, img in enumerate(row):
            ax = axs[row_idx, col_idx]
            img = F.to_pil_image(img.to("cpu"))
            ax.imshow(np.asarray(img), **imshow_kwargs)
            ax.set(xticklabels=[], yticklabels=[], xticks=[], yticks=[])
```

```
plt.tight_layout()
```

✓ Reading Videos Using Torchvision

We will first read a video using :func:`~torchvision.io.read\_video`. Alternatively one can use the new :class:`~torchvision.io.VideoReader` API (if torchvision is built from source). The video we will use here is free of use from pexels.com, credits go to [Pavel Danilyuk](#).

```
import tempfile
from pathlib import Path
from urllib.request import urlretrieve

video_url = "https://download.pytorch.org/tutorial/pexelscom_pavel_danilyuk_bas
video_path = Path(tempfile.mkdtemp()) / "basketball.mp4"
_ = urlretrieve(video_url, video_path)
```

:func:`~torchvision.io.read\_video` returns the video frames, audio frames and the metadata associated with the video. In our case, we only need the video frames.

Here we will just make 2 predictions between 2 pre-selected pairs of frames, namely frames (100, 101) and (150, 151). Each of these pairs corresponds to a single model input.

```
!pip install av
```

```
Requirement already satisfied: av in /usr/local/lib/python3.12/dist-packages (17
```

```
import torchvision
import importlib

# Đảm bảo av đã được nhận diện
try:
    ... import av
except ImportError:
    ... !pip install av
    ... import av

# Buộc torchvision kiểm tra lại sự hiện diện của av
import torchvision.io.video
importlib.reload(torchvision.io.video)
from torchvision.io import read_video
```

```
# Đọc video
frames, _, _ = read_video(str(video_path), output_format="TCHW", pts_unit="sec")

img1_batch = torch.stack([frames[100], frames[150]])
img2_batch = torch.stack([frames[101], frames[151]])

plot(img1_batch)
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/io/_video_deprecation_warnings.warn(
```



```
import sys
import subprocess

# Cài đặt av nếu chưa có
subprocess.check_call([sys.executable, "-m", "pip", "install", "av"])

# Reload lại các module liên quan để torchvision nhận diện được PyAV
import importlib
import torchvision.io.video
importlib.reload(torchvision.io.video)

print("Đã cài đặt và tải lại module av thành công. Bây giờ bạn hãy chạy lại ô n")

Đã cài đặt và tải lại module av thành công. Bây giờ bạn hãy chạy lại ô mã read_v
```

The RAFT model accepts RGB images. We first get the frames from `:func:~torchvision.io.read_video` and resize them to ensure their dimensions are divisible by 8. Note that we explicitly use `antialias=False`, because this is how those models were trained. Then we use the transforms bundled into the weights in order to preprocess the input and rescale its values to the required `[-1, 1]` interval.

```
from torchvision.models.optical_flow import Raft_Large_Weights

weights = Raft_Large_Weights.DEFAULT
transforms = weights.transforms()
```

```
def preprocess(img1_batch, img2_batch):
    img1_batch = F.resize(img1_batch, size=[520, 960], antialias=False)
    img2_batch = F.resize(img2_batch, size=[520, 960], antialias=False)
    return transforms(img1_batch, img2_batch)

img1_batch, img2_batch = preprocess(img1_batch, img2_batch)

print(f"shape = {img1_batch.shape}, dtype = {img1_batch.dtype}")

shape = torch.Size([2, 3, 520, 960]), dtype = torch.float32
```

✓ Estimating Optical flow using RAFT

We will use our RAFT implementation from

:func:~torchvision.models.optical_flow.raft_large, which follows the same architecture as the one described in the [original paper](#). We also provide the :func:~torchvision.models.optical_flow.raft_small model builder, which is smaller and faster to run, sacrificing a bit of accuracy.

```
from torchvision.models.optical_flow import raft_large

# If you can, run this example on a GPU, it will be a lot faster.
device = "cuda" if torch.cuda.is_available() else "cpu"

model = raft_large(weights=Raft_Large_Weights.DEFAULT, progress=False).to(device)
model = model.eval()

list_of_flows = model(img1_batch.to(device), img2_batch.to(device))
print(f"type = {type(list_of_flows)}")
print(f"length = {len(list_of_flows)} = number of iterations of the model")

Downloading: "https://download.pytorch.org/models/raft_large_C_T_SKHT_V2-ff5fad
type = <class 'list'>
length = 12 = number of iterations of the model
```

The RAFT model outputs lists of predicted flows where each entry is a (N, 2, H, W) batch of predicted flows that corresponds to a given "iteration" in the model. For more details on the iterative nature of the model, please refer to the [original paper](#). Here, we are only interested in the final predicted flows (they are the most accurate ones), so we will just retrieve the last item in the list.

As described above, a flow is a tensor with dimensions (2, H, W) (or (N, 2, H, W) for batches of flows) where each entry corresponds to the horizontal and vertical displacement of each pixel from the first image to the second image. Note that the predicted flows are in "pixel" unit, they are not normalized w.r.t. the dimensions of the images.

```
predicted_flows = list_of_flows[-1]
print(f"dtype = {predicted_flows.dtype}")
print(f"shape = {predicted_flows.shape} = (N, 2, H, W)")
print(f"min = {predicted_flows.min()}, max = {predicted_flows.max()}")
```

```
dtype = torch.float32
shape = torch.Size([2, 2, 520, 960]) = (N, 2, H, W)
min = -3.9027090072631836, max = 6.406130790710449
```

✓ Visualizing predicted flows

Torchvision provides the :func:`~torchvision.utils.flow_to_image` utility to convert a flow into an RGB image. It also supports batches of flows. each "direction" in the flow will be mapped to a given RGB color. In the images below, pixels with similar colors are assumed by the model to be moving in similar directions. The model is properly able to predict the movement of the ball and the player. Note in particular the different predicted direction of the ball in the first image (going to the left) and in the second image (going up).

```
from torchvision.utils import flow_to_image

flow_imgs = flow_to_image(predicted_flows)

# The images have been mapped into [-1, 1] but for plotting we want them in [0, 1]
img1_batch = [(img1 + 1) / 2 for img1 in img1_batch]

grid = [[img1, flow_img] for (img1, flow_img) in zip(img1_batch, flow_imgs)]
plot(grid)
```



✓ Bonus: Creating GIFs of predicted flows

In the example above we have only shown the predicted flows of 2 pairs of frames. A fun way to apply the Optical Flow models is to run the model on an entire video, and create a new video from all the predicted flows. Below is a snippet that can get you started with this. We comment out the code, because this example is being rendered on a machine without a GPU, and it would take too long to run it.

```
# from torchvision.io import write_jpeg
# for i, (img1, img2) in enumerate(zip(frames, frames[1:])):
#     # Note: it would be faster to predict batches of flows instead of indic
#     img1, img2 = preprocess(img1, img2)

#     list_of_flows = model(img1.to(device), img2.to(device))
#     predicted_flow = list_of_flows[-1][0]
#     flow_img = flow_to_image(predicted_flow).to("cpu")
#     output_folder = "/tmp/" # Update this to the folder of your choice
#     write_jpeg(flow_img, output_folder + f"predicted_flow_{i}.jpg")
```

Once the .jpg flow images are saved, you can convert them into a video or a GIF using ffmpeg with e.g.:

```
ffmpeg -f image2 -framerate 30 -i predicted_flow_%d.jpg -loop -1 flow.gif
```

